User-facing Feature	Database Queries
Login (for customer, booking agent, or staff login) - *replace with BookingAgent or AirlineStaff*	SELECT email, password FROM Customer WHERE email = %s
Customer registration 1. Checks whether customer email already exists 2. Inserts customer info into customer profile 3. Inserts customer phone number	SELECT email FROM Customer WHERE email = %s INSERT INTO Customer (email, name, password, building_num, street, city, state, passport_num, passport_exp_date, passport_country, date_of_birth) VALUES (...) INSERT INTO CustomerPhone (customer_email, phone_num) VALUES (...)
Register booking agent 1. Checks whether agent email already exists 2. Inserts agent info into database	SELECT email FROM BookingAgent WHERE email = %s INSERT INTO BookingAgent (email, password) VALUES (%s, %s)
Register airline staff 1. Check whether airline exists 2. Check whether staff username already exists 3. Inserts info into staff account 4. Inserts permissions into staff permission	SELECT name FROM Airline WHERE LOWER(name) = LOWER(%s) SELECT username FROM AirlineStaff WHERE username = %s INSERT INTO AirlineStaff (username, password, airline_name, first_name, last_name, date_of_birth) VALUES (...) INSERT INTO StaffPermissions (airline_name, staff_username, permissions) VALUES (...)
Search flights ● Main flight search ● Appending airport filters ● Appending date filter ● Sort and limit results	SELECT f.flight_num, f.airline_name, f.airplane_id, f.departure_airport, dep.city AS departure_city, f.arrival_airport, arr.city AS arrival_city, f.departure_time,

	f.arrival_time, f.price, f.flight_status FROM Flight AS f JOIN Airport AS dep ON f.departure_airport = dep.name JOIN Airport AS arr ON f.arrival_airport = arr.name WHERE f.departure_time > NOW() AND (LOWER(f.departure_airport) = LOWER(%s) OR LOWER(dep.city) = LOWER(%s)) AND (LOWER(f.arrival_airport) = LOWER(%s) OR LOWER(arr.city) = LOWER(%s)) AND DATE(f.departure_time) = %s ORDER BY f.departure_time ASC LIMIT 10
Queries available seat class for each returned flight result	SELECT sc.seat_class, sc.capacity, sc.price_factor, COALESCE(sold_counts.sold, 0) AS sold FROM SeatClass AS sc LEFT JOIN (SELECT seat_class, COUNT(*) AS sold FROM Ticket WHERE flight_num = %s GROUP BY seat_class) AS sold_counts ON LOWER(sold_counts.seat_class) = LOWER(sc.seat_class) WHERE sc.airplane_id = %s AND sc.capacity > COALESCE(sold_counts.sold, 0) ORDER BY ...
Check in-progress flights ● Only returns a flight if its status is exactly “in-progress”	SELECT f.flight_num, f.airline_name, f.departure_airport, dep.city AS departure_city, f.arrival_airport, arr.city AS arrival_city, f.departure_time, f.arrival_time, f.flight_status FROM Flight AS f JOIN Airport AS dep ON f.departure_airport = dep.name

	JOIN Airport AS arr ON f.arrival_airport = arr.name WHERE LOWER(f.airline_name) = LOWER(%s) AND UPPER(f.flight_num) = UPPER(%s) AND f.flight_status = 'in-progress'
Customer views their purchased flights ● Default “My Flights” view ● Appending optional filters ● Ordering results by departure time	SELECT f.flight_num, f.airline_name, f.departure_airport, f.arrival_airport, f.departure_time, f.arrival_time, t.price_charged FROM Ticket t JOIN Flight f ON t.flight_num = f.flight_num WHERE t.customer_email = %s AND DATE(f.departure_time) >= %s AND DATE(f.departure_time) <= %s AND f.departure_airport = %s AND f.arrival_airport = %s ORDER BY f.departure_time ASC
Customer directly purchases ticket 1. Gets the airplane for the flight 2. Checks whether customer already has a ticket for that flight 3. Verifies the selected seat class exists for that airplane 4. Counts how many seats in that class are already sold 5. Gets base flight price 6. Generates next ticket ID 7. Inserts the purchased ticket into the database	SELECT airplane_id FROM Flight WHERE flight_num = %s SELECT ticket_id FROM Ticket WHERE customer_email = %s AND flight_num = %s SELECT capacity, price_factor FROM SeatClass WHERE airplane_id = %s AND seat_class = %s SELECT COUNT(*) AS sold FROM Ticket WHERE flight_num = %s AND seat_class = %s SELECT price FROM Flight WHERE flight_num = %s SELECT MAX(ticket_id) AS max_id FROM Ticket INSERT INTO Ticket (ticket_id, flight_num, customer_email, airplane_id, seat_class, price_charged, purchase_date) VALUES (..., CURDATE())

Review customer spending ● Total spending over a selected/default range ● Monthly spending grouped by month	SELECT COALESCE(SUM(price_charged), 0) AS total_spending FROM Ticket WHERE customer_email = %s AND purchase_date BETWEEN %s AND %s SELECT DATE_FORMAT(purchase_date, '%Y-%m') AS month_key, COALESCE(SUM(price_charged), 0) AS amount FROM Ticket WHERE customer_email = %s AND purchase_date BETWEEN %s AND %s GROUP BY DATE_FORMAT(purchase_date, '%Y-%m') ORDER BY month_key
Agent views flights they've booked ● Default view of flights they've booked for customers ● Optional filters	SELECT f.*, t.customer_email, t.price_charged FROM Ticket t JOIN Flight f ON t.flight_num = f.flight_num WHERE t.agent_email = %s AND DATE(f.departure_time) >= %s AND DATE(f.departure_time) <= %s AND f.departure_airport = %s AND f.arrival_airport = %s
Agent opens the flight booking page (agent-only) 1. Fetch the selected flight details 2. Verify the agent is authorized to book for this airline 3. Fetch available seat classes for the flight's airplane	SELECT f.flight_num, f.airline_name, f.airplane_id, f.departure_airport, dep.city AS departure_city, f.arrival_airport, arr.city AS arrival_city, f.departure_time, f.arrival_time, f.price, f.flight_status FROM Flight AS f JOIN Airport AS dep ON f.departure_airport = dep.name JOIN Airport AS arr ON f.arrival_airport = arr.name WHERE f.flight_num = %s SELECT agent_email FROM AuthorizedFor WHERE agent_email = %s AND airline_name = %s SELECT sc.seat_class, sc.capacity, sc.price_factor,

	COALESCE(sold_counts.sold, 0) AS sold FROM SeatClass AS sc LEFT JOIN (SELECT seat_class, COUNT(*) AS sold FROM Ticket WHERE flight_num = %s GROUP BY seat_class) AS sold_counts ON LOWER(sold_counts.seat_class) = LOWER(sc.seat_class) WHERE sc.airplane_id = %s AND sc.capacity > COALESCE(sold_counts.sold, 0) ORDER BY ...
Agent purchases ticket for customer 1. Confirm the customer exists 2. Fetch the flight 3. Prevent duplicate ticket for the same customer/flight 4. Confirm agent is authorized for the flight's airline 5. Validate the seat class 6. Count sold seats in that class 7. Generate a unique ticket ID 8. Insert the ticket with both customer_email and agent_email	SELECT email FROM Customer WHERE email = %s SELECT flight_num, airline_name, airplane_id, price, departure_time FROM Flight WHERE flight_num = %s SELECT ticket_id FROM Ticket WHERE customer_email = %s AND flight_num = %s SELECT agent_email FROM AuthorizedFor WHERE agent_email = %s AND airline_name = %s SELECT capacity, price_factor FROM SeatClass WHERE airplane_id = %s AND LOWER(seat_class) = LOWER(%s) SELECT COUNT(*) AS sold FROM Ticket WHERE flight_num = %s AND LOWER(seat_class) = LOWER(%s) SELECT COALESCE(MAX(ticket_id), 0) + 1 AS next_id FROM Ticket INSERT INTO Ticket (ticket_id, flight_num, customer_email, agent_email, airplane_id, seat_class, price_charged, purchase_date) VALUES (..., CURDATE())
Agent views commission and top customers	SELECT COALESCE(SUM(price_charged * 0.10), 0) AS

<table>
<tr>
<td>

- 30 day summary
- Top 5 customers by ticket count in the last six months
- Top 5 customers by commission earned in last 12 months

</td>
<td>

```
total_commission,
   COALESCE(AVG(price_charged * 0.10), 0) AS
avg_commission,
   COUNT(*) AS tickets
FROM Ticket
WHERE agent_email = %s
  AND purchase_date >= DATE_SUB(CURDATE(),
INTERVAL 30 DAY)

SELECT customer_email, COUNT(*) AS count
FROM Ticket
WHERE agent_email = %s
  AND purchase_date >= DATE_SUB(CURDATE(),
INTERVAL 6 MONTH)
GROUP BY customer_email
ORDER BY count DESC
LIMIT 5

SELECT customer_email, SUM(price_charged *
0.10) AS total
FROM Ticket
WHERE agent_email = %s
  AND purchase_date >= DATE_SUB(CURDATE(),
INTERVAL 12 MONTH)
GROUP BY customer_email
ORDER BY total DESC
LIMIT 5
```

</td>
</tr>
<tr>
<td>

Airline staff loads staff identity and permissions

1. Gets specific row in AirlineStaff that corresponds to a specific staff member
2. Grab formal airline name
3. Fetch permissions of that staff member

</td>
<td>

```
SELECT username, airline_name, first_name,
last_name
FROM AirlineStaff
WHERE username = %s

SELECT name
FROM Airline
WHERE LOWER(name) = LOWER(%s)

SELECT permissions
FROM StaffPermissions
WHERE LOWER(airline_name) = LOWER(%s)
  AND staff_username = %s
```

</td>
</tr>
<tr>
<td>

Staff dashboard

- Grabs flights for the staff's airline, this is the default view
- Filter to show flights from now through the next 30 days
- Appending optional date and route filters

</td>
<td>

```
SELECT
   f.flight_num,
   f.airline_name,
   f.departure_airport,
   dep.city AS departure_city,
   f.arrival_airport,
   arr.city AS arrival_city,
   f.departure_time,
   f.arrival_time,
   f.price,
   f.flight_status,
   f.airplane_id
FROM Flight AS f
JOIN Airport AS dep
```

</td>
</tr>
</table>

	ON dep.name = f.departure_airport JOIN Airport AS arr ON arr.name = f.arrival_airport WHERE LOWER(f.airline_name) = LOWER(%s) AND f.departure_time >= NOW() AND f.departure_time < DATE_ADD(NOW(), INTERVAL 30 DAY) AND DATE(f.departure_time) >= %s AND DATE(f.departure_time) <= %s AND (f.departure_airport = %s OR dep.city = %s) AND (f.arrival_airport = %s OR arr.city = %s) ORDER BY f.departure_time ASC
Staff views flights taken by a specific customer 1. Confirm the customer exists 2. Fetch the customer's tickets on this airline	SELECT email, name FROM Customer WHERE email = %s SELECT f.flight_num, f.airline_name, f.departure_airport, f.arrival_airport, f.departure_time, f.arrival_time, f.flight_status, t.ticket_id, t.seat_class, t.price_charged, t.purchase_date FROM Ticket AS t JOIN Flight AS f ON f.flight_num = t.flight_num WHERE t.customer_email = %s AND LOWER(f.airline_name) = LOWER(%s) ORDER BY f.departure_time DESC
Staff views passenger list for one flight 1. Confirm the flight belongs to the staff member's airline 2. Fetch passengers on that flight	SELECT flight_num, airline_name, departure_airport, arrival_airport, departure_time FROM Flight WHERE flight_num = %s AND LOWER(airline_name) = LOWER(%s) SELECT c.email, c.name, c.passport_num, c.passport_country, t.ticket_id, t.seat_class, t.price_charged, t.purchase_date, t.agent_email FROM Ticket AS t JOIN Customer AS c

	ON c.email = t.customer_email WHERE t.flight_num = %s ORDER BY c.name, c.email
Staff views airline-wide analytics and stats ● Top booking agents for the airline this month/year by tickets and commission ● Most frequent customer in the last year ● Tickets sold per month ● Delay vs on-time stats ● Top destinations in the last 3 months/year	SELECT t.agent_email, COUNT(*) AS tickets_sold, COALESCE(SUM(t.price_charged * 0.10), 0) AS commission FROM Ticket AS t JOIN Flight AS f ON f.flight_num = t.flight_num WHERE LOWER(f.airline_name) = LOWER(%s) AND t.agent_email IS NOT NULL AND ... GROUP BY t.agent_email ORDER BY ... LIMIT 5 SELECT t.customer_email, c.name, COUNT(*) AS tickets FROM Ticket AS t JOIN Flight AS f ON f.flight_num = t.flight_num JOIN Customer AS c ON c.email = t.customer_email WHERE LOWER(f.airline_name) = LOWER(%s) AND t.purchase_date >= DATE_SUB(CURDATE(), INTERVAL 1 YEAR) GROUP BY t.customer_email, c.name ORDER BY tickets DESC LIMIT 1 SELECT DATE_FORMAT(t.purchase_date, '%Y-%m') AS month_key, COUNT(*) AS tickets FROM Ticket AS t JOIN Flight AS f ON f.flight_num = t.flight_num WHERE LOWER(f.airline_name) = LOWER(%s) AND t.purchase_date >= DATE_SUB(CURDATE(), INTERVAL 12 MONTH) GROUP BY DATE_FORMAT(t.purchase_date, '%Y-%m') ORDER BY month_key SELECT SUM(CASE WHEN flight_status = 'delayed' THEN 1 ELSE 0 END) AS delayed_count, SUM(CASE WHEN flight_status = 'on-time' THEN 1 ELSE 0 END) AS on_time_count, COUNT(*) AS total_flights FROM Flight

	WHERE LOWER(airline_name) = LOWER(%s) SELECT f.arrival_airport, a.city, COUNT(*) AS ticket_count FROM Ticket AS t JOIN Flight AS f ON f.flight_num = t.flight_num JOIN Airport AS a ON a.name = f.arrival_airport WHERE LOWER(f.airline_name) = LOWER(%s) AND t.purchase_date >= ... GROUP BY f.arrival_airport, a.city ORDER BY ticket_count DESC LIMIT 5
Admin creates a new airport 1. Check for duplicate airport code 2. Insert newly created airport into database	SELECT name FROM Airport WHERE name = %s INSERT INTO Airport (name, city) VALUES (%s, %s)
Admin adds an airplane and its seat class capacity 1. Check for duplicate airplane ID 2. Insert airplane into database 3. Insert one SeatClass row per positive-capacity class	SELECT id FROM Airplane WHERE id = %s INSERT INTO Airplane (id, airline_name, seat_capacity) VALUES (%s, %s, %s) INSERT INTO SeatClass (airplane_id, seat_class, capacity, price_factor) VALUES (%s, %s, %s, %s)
Admin creates a new flight 1. Check for duplicate flight number 2. Verify the airplane belongs to the staff airline 3. Verify both airports exist 4. Insert new flight into database	SELECT flight_num FROM Flight WHERE UPPER(flight_num) = UPPER(%s) SELECT id FROM Airplane WHERE id = %s AND LOWER(airline_name) = LOWER(%s) SELECT name FROM Airport WHERE UPPER(name) = UPPER(%s) INSERT INTO Flight (flight_num, airplane_id, airline_name, departure_airport, arrival_airport, departure_time, arrival_time, price, flight_status) VALUES (...)

Admin authorizes booking agent for their airline 1. Confirm booking agent exists 2. Check whether that authorization already exists 3. Insert authorization into database	SELECT email FROM BookingAgent WHERE email = %s SELECT agent_email FROM AuthorizedFor WHERE agent_email = %s AND airline_name = %s INSERT INTO AuthorizedFor (agent_email, airline_name) VALUES (%s, %s)
Operator changes a flight's status 1. Fetch the flight and confirm it belongs to the operator's airline 2. Update the status in the database	SELECT flight_num, airline_name, departure_airport, arrival_airport, departure_time, arrival_time, flight_status FROM Flight WHERE flight_num = %s AND airline_name = %s UPDATE Flight SET flight_status = %s WHERE flight_num = %s AND airline_name = %s